Text Mode

2. Flowchart

การทำงานของโปรแกรมตามโจทย์ สามารถอธิบายขั้นตอนและตรรกะในแต่ละส่วนได้ดังนี้

1. การออกแบบคลาส Student

- การกำหนดแอตทริบิวต์ (Constructor): มีการสร้างคลาสชื่อ Student โดยภายในฟังก์ชัน __init__ มีการรับพารามิเตอร์เพื่อกำหนดค่าให้กับตัวแปรประจำตัวของนักเรียน ได้แก่ รหัสนักศึกษา (student_id), ชื่อ (name), หลักสูตรหรือวิชาเรียน (course) และอายุ (age)
- เมธอดแสดงผล (show): ภายในคลาสมีเมธอดชื่อ show() สำหรับดึงข้อมูลแอตทริบิวต์ของนักเรียนคนนั้นๆ ออกมาพิมพ์เป็นข้อความสรุปรายละเอียดอย่างครบถ้วน ไม่ว่าจะเป็นรหัส ชื่อ อายุ และวิชาที่ลงทะเบียน

2. การสร้างฟังก์ชันค้นหานักเรียนที่มีอายุน้อยที่สุด (show_youngest)

- ฟังก์ชันนี้ทำหน้าที่รับอาร์กิวเมนต์เป็นอาเรย์ของอ็อบเจกต์ Student เข้ามาตรวจสอบ
- การป้องกันข้อผิดพลาด: เริ่มต้นด้วยการเช็กว่ามีข้อมูลในลิสต์หรือไม่ หากไม่มีข้อมูลจะแจ้งเตือนและสิ้นสุดการทำงานทันที
- ตรรกะการเปรียบเทียบ: กำหนดให้นักเรียนคนแรกในลิสต์เป็นตัวแทนที่มีอายุน้อยที่สุดตั้งต้นไว้ก่อน จากนั้นใช้การวนลูปตรวจสอบนักเรียนทีละคนในลิสต์ โดยนำค่า age ของแต่ละคนมาเปรียบเทียบ หากพบใครที่มีอายุน้อยกว่าตัวแทนปัจจุบัน ก็จะอัปเดตให้อ็อบเจกต์คนนั้นกลายเป็นผู้ที่มีอายุน้อยที่สุดแทน
- การแสดงผล: เมื่อวนลูปครบทุกคนแล้ว จะเรียกใช้เมธอด show() ของอ็อบเจกต์นักเรียนที่อายุน้อยที่สุดคนนั้น เพื่อแสดงผลข้อมูลตามที่โจทย์กำหนด

3. กระบวนการอ่านไฟล์และการทำงานหลัก (main)

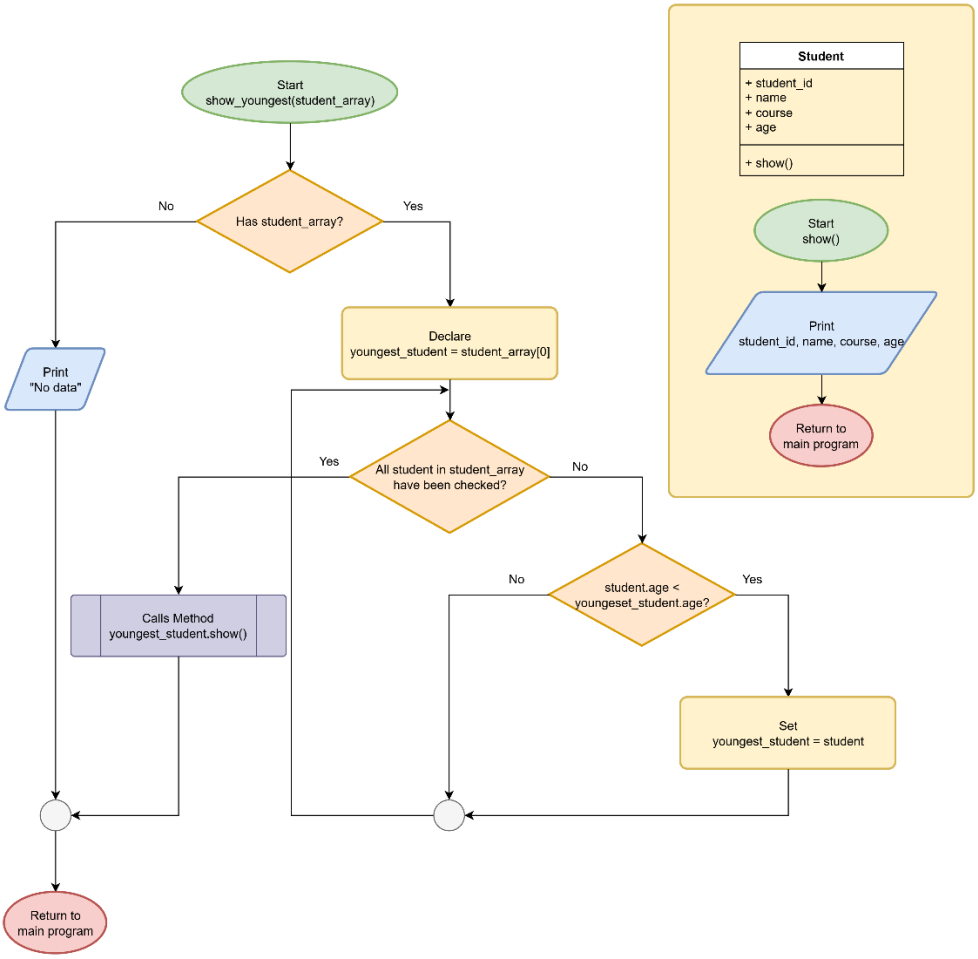
- การอ่านข้อมูลจากไฟล์: มีการเปิดไฟล์ข้อความ (data.txt) ภายใต้บล็อกดักจับข้อผิดพลาด (Try-Except) เพื่อป้องกันกรณีไม่พบไฟล์

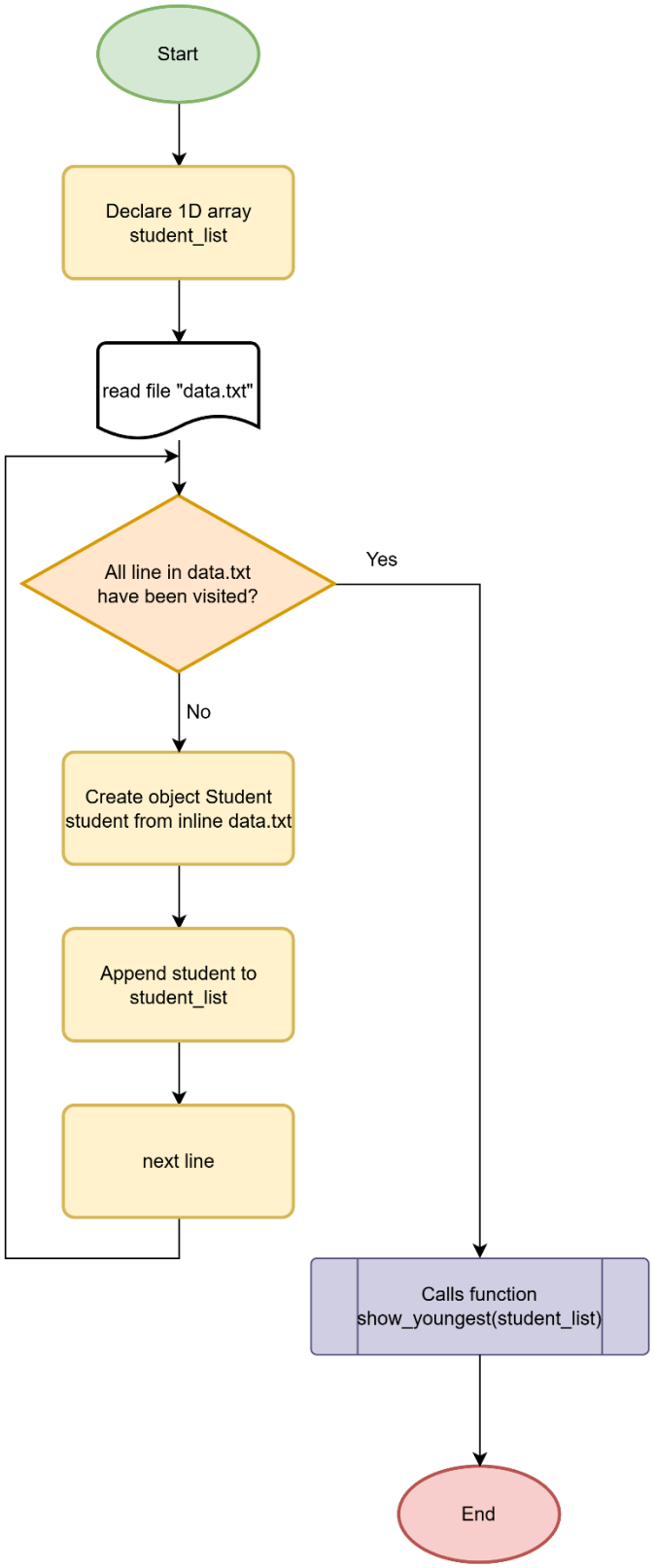
- การแยกข้อมูลและสร้างอ็อบเจกต์: โปรแกรมจะวนอ่านทีละบรรทัด พร้อมทำการตัดช่องว่าง/การขึ้นบรรทัดใหม่ และแยกข้อความด้วยเครื่องหมายจุลภาค ทำให้ได้ข้อมูล 4 ส่วน

(รหัส, ชื่อ, วิชา, อายุ) จากนั้นนำข้อมูลทั้ง 4 ตัวไปสร้างเป็นอ็อบเจกต์ Student และนำไปเก็บสะสมไว้ในลิสต์หลัก

- การส่งต่อข้อมูล: เมื่ออ่านข้อมูลจากไฟล์จนครบและได้อาเรย์ของอ็อบเจกต์ Student สมบูรณ์แล้ว โปรแกรมจะส่งลิสต์ดังกล่าวไปยังฟังก์ชัน show_youngest เพื่อทำการค้นหาและพิมพ์ข้อมูลของนักเรียนที่อายุน้อยที่สุดออกมาแสดงบนหน้าจอ

Flowchart




Start
Declare 1D array
student_list
read file "data.txt"
All line in data.txt
have been visited?
Yes
No
Create object Student
student from inline data.txt
Append student to
student_list
next line
Calls function
show_youngest(student_list)
End

Program

```python
class Student:
    def __init__(self, id, name, course, age):
        self.student_id: str = id
        self.name: str = name
        self.course: str = course
        self.age: int = age

    def show(self):
        print(f"{self.student_id} {self.name}, age {str(self.age)}. Is enrolled in {self.course}.")

def show_youngest(student_array):
    if not student_array:
        print("No data")
        return

    youngest_student = student_array[0]
    for student in student_array:
        if student.age < youngest_student.age:
            youngest_student = student

    youngest_student.show()
```

```python
def main():
    student_list = []
    file_name: str = "data.txt"

    try:
        with open(file_name, "r", encoding="utf-8") as file:
            for lines in file:
                data = lines.strip().split(", ")

                if len(data) == 4:
                    stud_id, name, course, age = data
                    student = Student(stud_id, name, course, age)
                    student_list.append(student)
    except:
        print("File not found.")

    print("----Youngest Student----")
    show_youngest(student_list)


if __name__ == "__main__":
    main()
```

data.txt

```
22189, Chanasin Na nakorn, Pre-cadet Academy, 18
22629, Suttipat Euaphattanakit, Mathematics, 18
22510, Phuriphat Wongphun, Accounting, 17
22321, Thitima Pakaew, Civil Law, 11
```

output

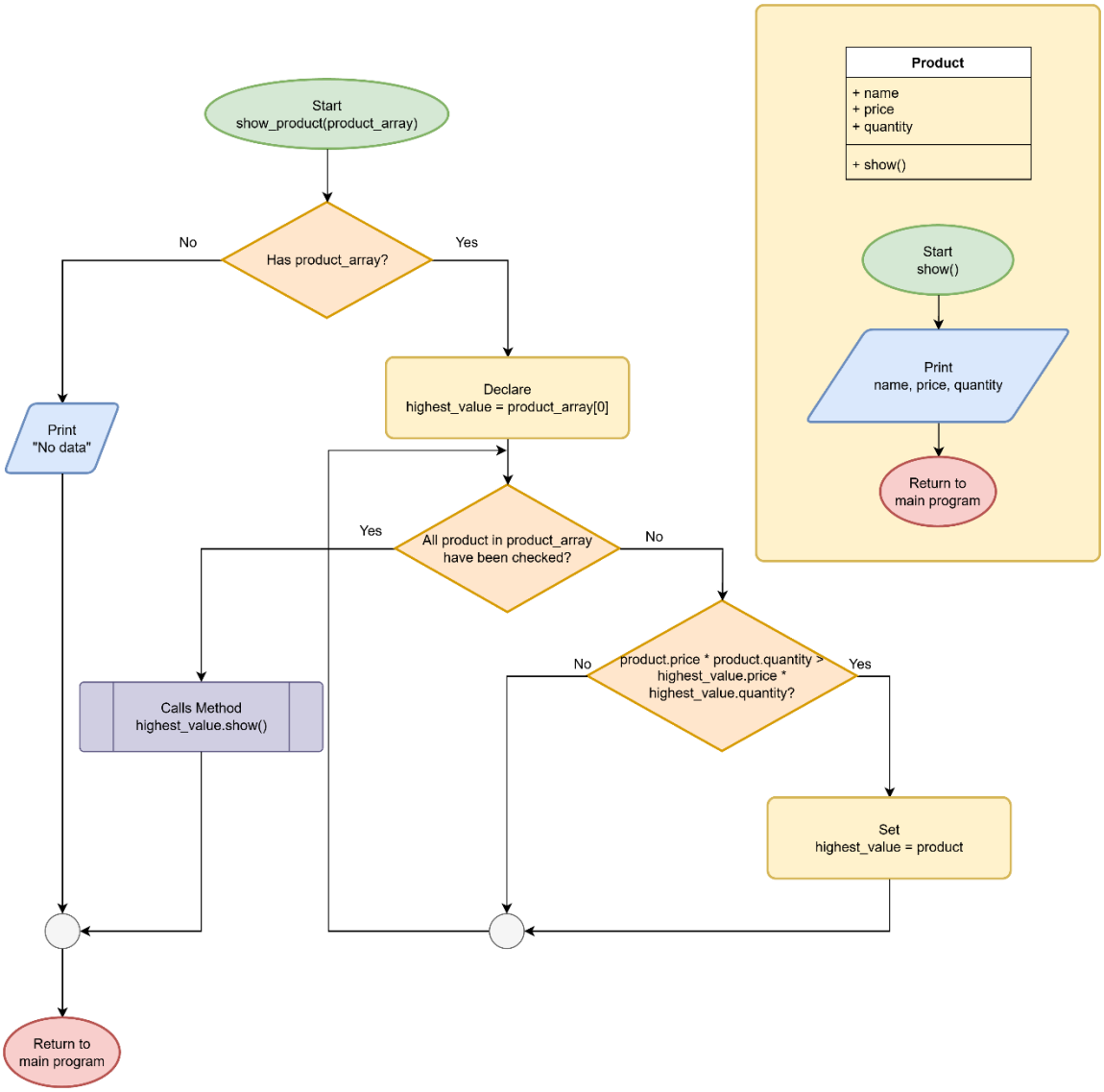
```
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

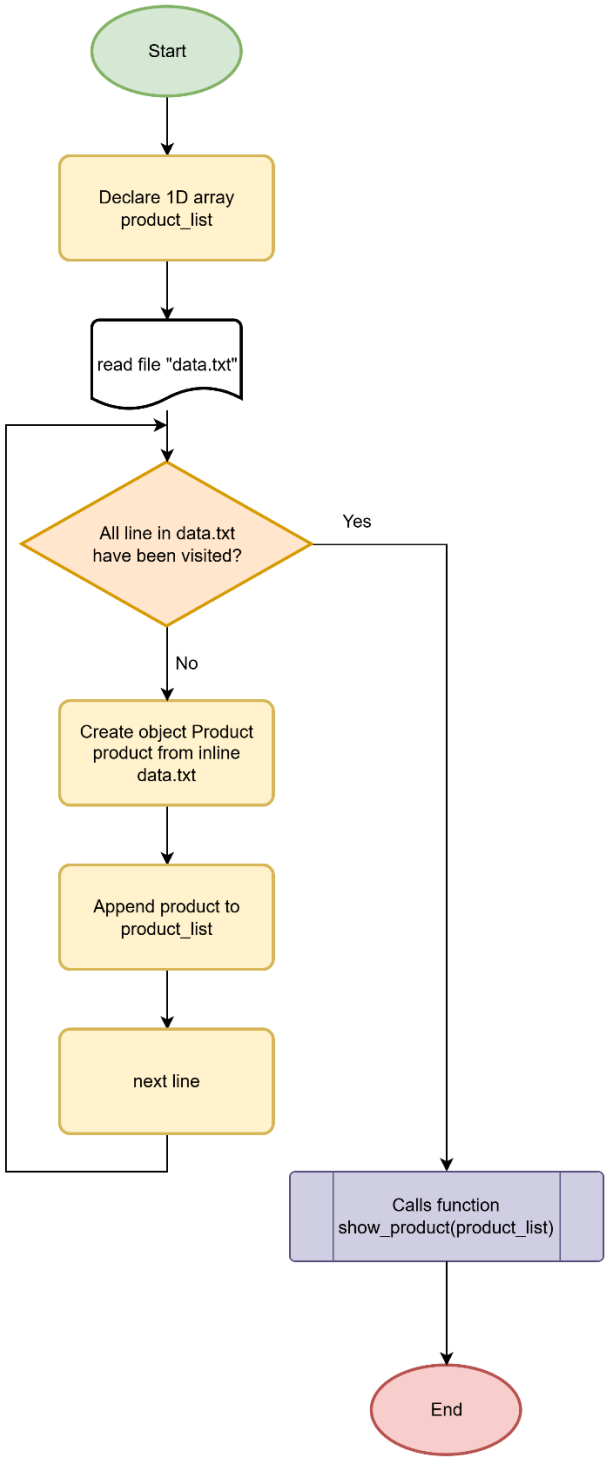
D:\KMUTNB\1-69\Computer-Programming\week8\Lab\6901012610544-Lab8-02>python 6901012610544-Lab8-02.py
----Youngest Student----
22321 Thitima Pakaew, age 11. Is enrolled in Civil Law.
```

3. การทำงานของโปรแกรมในข้อนี้ ใช้หลักการเดียวกันกับข้อที่ผ่านมาแต่ ในส่วนของการเปรียบเทียบค่าจะใช้การตรวจสอบ

$$product.price * product.quantity > highest_value.price * highest_value.quantity$$

Flowchart




Start
Declare 1D array product_list
read file "data.txt"
All line in data.txt have been visited?
Yes
No
Create object Product product from inline data.txt
Append product to product_list
next line
Calls function show_product(product_list)
End

Program

```python
class Product:
    def __init__(self, name, price, quantity):
        self.name: str = name
        self.price: int = price
        self.quantity: int = quantity

    def show(self):
        print(f"Product: {self.name}\nPrice: {str(self.price)} thb\nQuantity: {str(self.quantity)} pcs\nValue: {int(self.price)*int(self.quantity)} thb")

def show_product(product_array):
    if not product_array:
        print("No data")
        return

    highest_value = product_array[0]
    for product in product_array:
        if int(product.price) * int(product.quantity) > int(highest_value.price) * int(highest_value.quantity):
            highest_value = product

    highest_value.show()

```

```python
def main():
    product_list = []
    file_name: str = "data.txt"

    try:
        with open(file_name, "r", encoding="utf-8") as file:
            for lines in file:
                data = lines.strip().split(", ")

                if len(data) == 3:
                    name, price, quantity = data
                    product = Product(name, price, quantity)
                    product_list.append(product)
    except:
        print("File not found.")

    print("----Highest Product Value----")
    show_product(product_list)


if __name__ == "__main__":
    main()
```

data.txt

```
Lays, 20, 500
CocalCola, 15, 340
PHL-03, 200, 3
```

output

```
D:\KMUTNB\1-69\Computer-Programming\week8\Lab\6901012610544-Lab8-03>python 6901012610544-Lab8-03.py
----Highest Product Value----
Product: Lays
Price: 20 thb
Quantity: 500 pcs
Value: 10000 thb
```

4. จากปฏิบัติการครั้งที่ 6 สามารถปรับเปลี่ยนโค้ดโดยใช้ Class ได้ดังนี้

```python
class Matrix:
    def __init__(self, data):
        """
        Initialize the matrix with a 2D list of numbers.
        """
        self.data = data
        self.rows = len(data)
        self.cols = len(data[0]) if self.rows > 0 else 0

    def show(self):
        """
        Display the matrix row by row with aligned formatting.
        """
        for row in self.data:
            print("[ " + "  ".join(f"{val:>4}" for val in row) + " ]")
        print()

    def add(self, other):
        """
        Add another matrix of the same dimensions element-wise.
        """
        #Validate that dimensions match
        if self.rows != other.rows or self.cols != other.cols:
            raise ValueError("Matrices must have the same dimensions to perform addition.")

        result = [
            [self.data[r][c] + other.data[r][c] for c in range(self.cols)]
            for r in range(self.rows)
        ]
        return Matrix(result)
```

```python
    def subtract(self, other):
        """
        Subtract another matrix of the same dimensions element-wise.
        """
        #Validate that dimensions match
        if self.rows != other.rows or self.cols != other.cols:
            raise ValueError("Matrices must have the same dimensions to perform subtraction.")

        result = [
            [self.data[r][c] - other.data[r][c] for c in range(self.cols)]
            for r in range(self.rows)
        ]
        return Matrix(result)

    def multiplication(self, other):
        """
        Multiply this matrix by another matrix using standard matrix multiplication.
        Requirement: Number of columns in Matrix A must equal number of rows in Matrix B.
        """
        #Validate matrix multiplication condition
        if self.cols != other.rows:
            raise ValueError(
                f"Cannot multiply: Columns of first matrix ({self.cols}) "
                f"must match rows of second matrix ({other.rows})."
            )

        #Result size: (rows of A) x (columns of B)
        result = [
            [
                sum(self.data[i][k] * other.data[k][j] for k in range(self.cols))
                for j in range(other.cols)
            ]
            for i in range(self.rows)
        ]
        return Matrix(result)
```

```
D:\KMUTNB\1-69\Computer-Programming\week8\Lab>python 6901012610544-Lab8-04.py
Matrix 1:
[    1     2     3 ]
[    4     5     6 ]

Matrix 2:
[    7     8     9 ]
[    1     2     3 ]

Matrix 1 + Matrix 2 (add):
[    8    10    12 ]
[    5     7     9 ]

Matrix 1 - Matrix 2 (subtract):
[   -6    -6    -6 ]
[    3     3     3 ]

Matrix 3:
[    1     2 ]
[    3     4 ]
[    5     6 ]

Matrix 1 * Matrix 3 (multiplication):
[   22    28 ]
[   49    64 ]
```

5.การทำงานของโปรแกรมในข้อนี้ สามารถอธิบายขั้นตอนและตรรกะในแต่ละส่วนได้ดังนี้ครับ

1. การกำหนดค่านามสกุลไฟล์รูปภาพ (Configuration)

- มีการนำเข้าโมดูลสำหรับจัดการระบบไฟล์และโฟลเดอร์ของระบบปฏิบัติการ
- กำหนดตัวแปรคงที่แบบ Tuple เพื่อรวบรวมนามสกุลไฟล์ของรูปภาพที่โปรแกรมยอมรับและต้องการกรองข้อมูล ได้แก่ นามสกุลไฟล์รูปภาพมาตรฐาน เช่น ไฟล์ภาพประเภท JPEG, PNG และ GIF

2. ฟังก์ชันแสดงรายการและขนาดของไฟล์ทั้งหมด (list_all_files)

- ฟังก์ชันนี้รับที่อยู่ของโฟลเดอร์ (Directory Path) เข้ามาประมวลผล
- การวนลูปตรวจสอบ: ทำการอ่านรายชื่อไฟล์และโฟลเดอร์ย่อยทั้งหมดที่อยู่ในโฟลเดอร์เป้าหมาย จากนั้นนำชื่อไฟล์มาเชื่อมต่อเข้ากับที่อยู่โฟลเดอร์เดิมเพื่อให้ได้เส้นทางไฟล์แบบเต็ม (Full Path) แล้วตรวจสอบว่ารายการนั้นเป็นไฟล์จริง ๆ หรือไม่
- การคำนวณและแสดงผล: หากเป็นไฟล์ โปรแกรมจะดึงขนาดของไฟล์นั้นมาบวกสะสมไว้ในตัวแปรขนาดรวม พร้อมนับจำนวนไฟล์เพิ่มขึ้นทีละหนึ่ง จากนั้นพิมพ์ชื่อไฟล์พร้อมขนาดของแต่ละไฟล์ออกมา และสรุปผลรวมจำนวนไฟล์ทั้งหมดรวมถึงขนาดรวมทั้งหมดในหน่วยไบต์และเมกะไบต์ในตอนท้าย

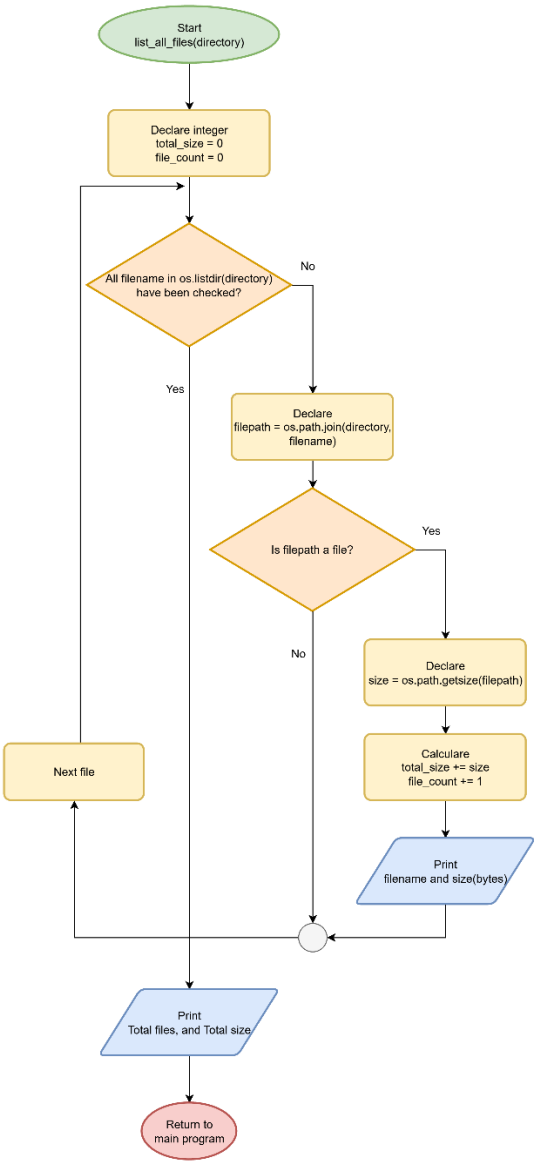
3. ฟังก์ชันกรองและแสดงเฉพาะไฟล์รูปภาพ (list_image_files)

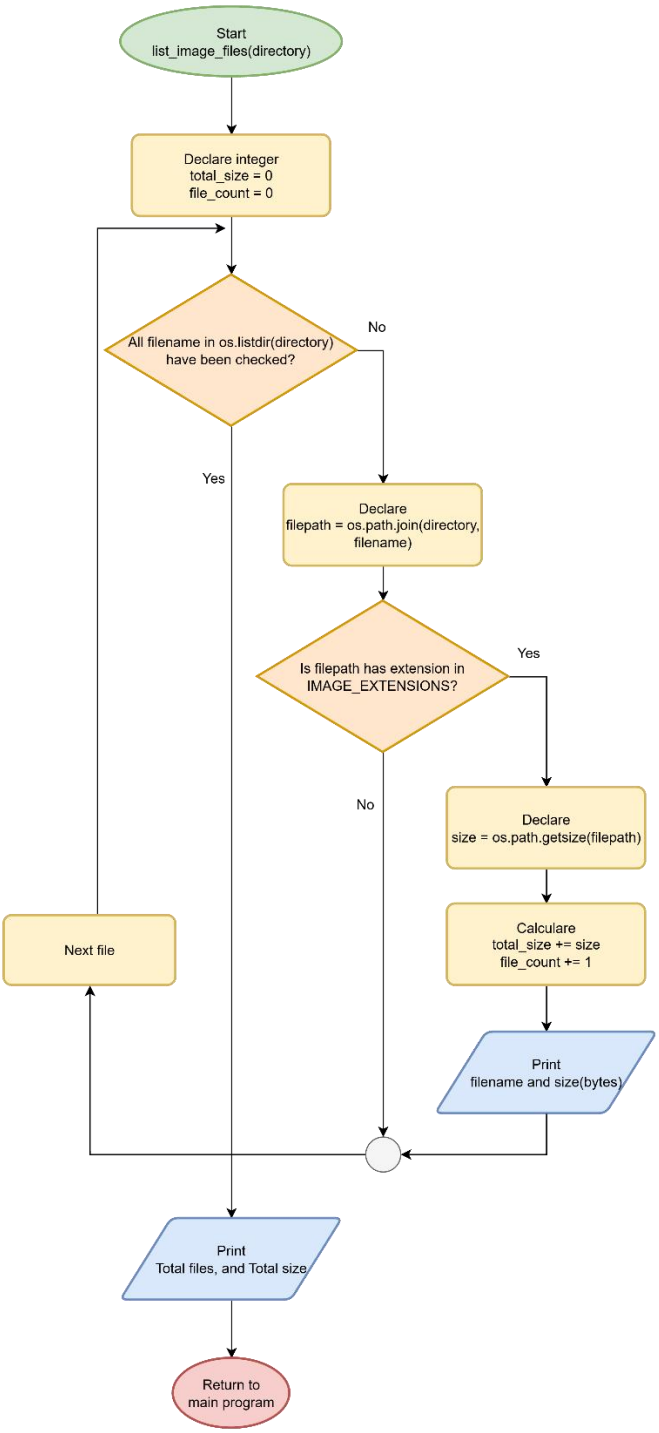
- ฟังก์ชันนี้มีหน้าที่ตรวจสอบโฟลเดอร์เดียวกันซ้ำอีกครั้ง แต่เพิ่มเงื่อนไขการคัดกรองเฉพาะไฟล์รูปภาพตามที่โจทย์กำหนด
- การตรวจสอบเงื่อนไข: นอกเหนือจากการตรวจสอบว่าเป็นไฟล์แล้ว ยังมีการตรวจสอบส่วนท้ายของชื่อไฟล์โดยปรับให้เป็นตัวพิมพ์เล็กก่อน เพื่อดูว่าลงท้ายด้วยนามสกุลรูปภาพที่กำหนดไว้หรือไม่
- การคำนวณและแสดงผล: หากเป็นไฟล์รูปภาพ โปรแกรมจะดึงขนาดไฟล์มาสะสมในตัวแปรผลรวมขนาดรูปภาพ และนับจำนวนไฟล์รูปภาพ พร้อมทั้งแสดงชื่อไฟล์และขนาดของแต่ละภาพ จากนั้นสรุปจำนวนรูปภาพทั้งหมดและขนาดไฟล์รูปภาพรวมทั้งหมดออกมาแสดงผล

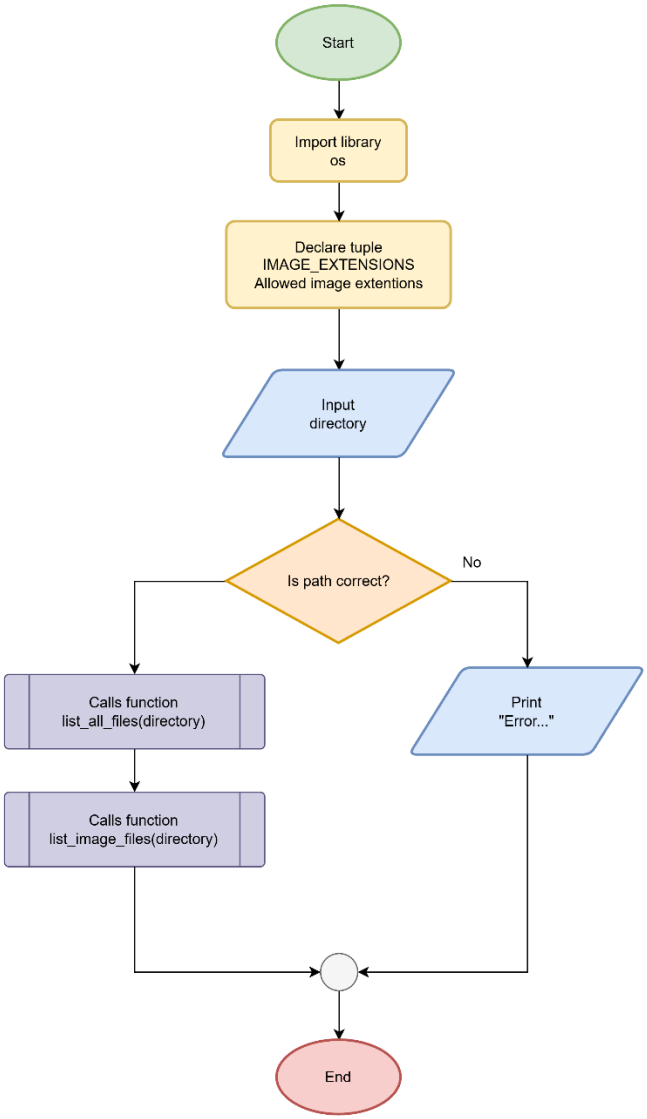
4. การควบคุมการทำงานหลัก (main)

- การรับข้อมูลและตรวจสอบ: โปรแกรมเริ่มต้นด้วยการรับข้อความที่อยู่ของโฟลเดอร์จากผู้ใช้งาน พร้อมตัดช่องว่างหน้าหลังทิ้ง และมีกระบวนการตรวจสอบความถูกต้องว่าเส้นทางดังกล่าวเป็นโฟลเดอร์ที่มีอยู่จริงหรือไม่ หากไม่ใช่จะแจ้งข้อผิดพลาดและยกเลิกการทำงานทันที
- การเรียกใช้งาน: หากเส้นทางโฟลเดอร์ถูกต้อง โปรแกรมจะเรียกใช้ฟังก์ชันแสดงรายการไฟล์ทั้งหมดก่อน จากนั้นจึงตามด้วยฟังก์ชันที่กรองเฉพาะไฟล์รูปภาพ ทำให้ได้ผลลัพธ์ทั้งสองส่วนตามความต้องการของโจทย์

Flowchart




Start
list_image_files(directory)
Declare integer
total_size = 0
file_count = 0
All filename in os.listdir(directory)
have been checked?
No
Yes
Declare
filepath = os.path.join(directory,
filename)
Is filepath has extension in
IMAGE_EXTENSIONS?
Yes
No
Declare
size = os.path.getsize(filepath)
Calculare
total_size += size
file_count += 1
Print
filename and size(bytes)
Next file
Print
Total files, and Total size
Return to
main program


Start
Import library
os
Declare tuple
IMAGE_EXTENSIONS
Allowed image extentions
Input
directory
Is path correct?
No
Calls function
list_all_files(directory)
Calls function
list_image_files(directory)
Print
"Error..."
End

Program

```python
import os

#Allowed image extensions
IMAGE_EXTENSIONS = ('.jpg', '.jpeg', '.png', '.gif')


def list_all_files(directory: str) -> None:
    """Lists every file in the directory and calculates total size."""
    print(f"\n=== All Files in '{directory}' ===")
    total_size = 0
    file_count = 0

    for filename in os.listdir(directory):
        filepath = os.path.join(directory, filename)
        if os.path.isfile(filepath):
            size = os.path.getsize(filepath)
            total_size += size
            file_count += 1
            print(f"- {filename} ({size:,} bytes)")

    print("-" * 45)
    print(f"Total files: {file_count}")
    print(f"Total file size: {total_size:,} bytes ({total_size / (1024 * 1024):.2f} MB)")
```

```python
def list_image_files(directory: str) -> None:
    """Lists only image files in the directory and calculates their total size."""
    print(f"\n=== Image Files in '{directory}' ===")
    total_size = 0
    image_count = 0

    for filename in os.listdir(directory):
        filepath = os.path.join(directory, filename)
        if os.path.isfile(filepath) and filename.lower().endswith(IMAGE_EXTENSIONS):
            size = os.path.getsize(filepath)
            total_size += size
            image_count += 1
            print(f"- {filename} ({size:,} bytes)")

    print("-" * 45)
    print(f"Total image files: {image_count}")
    print(f"Total image size: {total_size:,} bytes ({total_size / (1024 * 1024):.2f} MB)")


def main():
    directory = input("Enter directory path: ").strip()

    if not os.path.isdir(directory):
        print("Error: The specified path is not a valid directory.")
        return

    list_all_files(directory)
    list_image_files(directory)


if __name__ == "__main__":
    main()
```

ตัวอย่างไฟล์ที่ใช้

Konfuzius-1770.jpg	26/9/2569 19:31	JPG File	499 KB
Tao_Te_Ching.txt	26/9/2569 19:28	Text Document	4 KB
The_Analects_of_Confucius.txt	26/9/2569 19:29	Text Document	6 KB
Zhang_Lu-Laozi_Riding_an_Ox.jpg	26/9/2569 19:30	JPG File	780 KB

Output

```
D:\KMUTNB\1-69\Computer-Programming\week8\Lab\6901012610544-Lab8-05>python 6901012610544-Lab8-05.py
Enter directory path: .

=== All Files in '.' ===
- 6901012610544-Lab8-05.py (1,808 bytes)
- Konfuzius-1770.jpg (510,850 bytes)
- Tao_Te_Ching.txt (3,195 bytes)
- The_Analects_of_Confucius.txt (5,240 bytes)
- Zhang_Lu-Laozi_Riding_an_Ox.jpg (798,644 bytes)
---------------------------------------------
Total files: 5
Total file size: 1,319,737 bytes (1.26 MB)

=== Image Files in '.' ===
- Konfuzius-1770.jpg (510,850 bytes)
- Zhang_Lu-Laozi_Riding_an_Ox.jpg (798,644 bytes)
---------------------------------------------
Total image files: 2
Total image size: 1,309,494 bytes (1.25 MB)
```

Graphic Mode

เครื่องมือที่ใช้: Python mode for Processing4 (Version 4.0a)

6. แบบไม่ใช้ Class

```
# ============================================================
# Program 1: Balloon Animation without Class
# ============================================================

# List storing initial data (attributes) for 5 balloons:
# x: horizontal position, y: vertical position, speed: rise rate, color: (R, G, B)
balloons = [
    {"x": 100, "y": 600, "speed": 2.0, "color": (255, 99, 132)},
    {"x": 200, "y": 650, "speed": 3.2, "color": (54, 162, 235)},
    {"x": 300, "y": 580, "speed": 1.5, "color": (255, 206, 86)},
    {"x": 400, "y": 620, "speed": 2.8, "color": (75, 192, 192)},
    {"x": 500, "y": 670, "speed": 2.2, "color": (153, 102, 255)}
]

def setup():
    size(600, 600)

def draw_balloon(x, y, col):
    """Draws a single balloon and its string at (x, y) with color col."""
    # Draw string
    stroke(120)
    strokeWeight(1.5)
    line(x, y + 38, x, y + 95)

    # Draw balloon body
    noStroke()
    fill(col[0], col[1], col[2])
    ellipse(x, y, 60, 76)

    # Draw balloon knot
    triangle(x - 5, y - 42, x + 5, y + 42, x, y - 37)
```

```
def draw():
    background(235, 245, 255)  # Sky blue background

    # Loop through each balloon to update animation and draw
    for b in balloons:
        # Move balloon upward
        b["y"] -= b["speed"]

        # Reset to bottom when floating off the top edge
        if b["y"] < -100:
            b["y"] = height + 50

        # Render balloon
        draw_balloon(b["x"], b["y"], b["color"])
```

แบบใช้ Class

```
# ============================================================
# Program 2: Balloon Animation with Class
# ============================================================

class Balloon:
    def __init__(self, x, y, speed, col):
        """Initialize balloon attributes."""
        self.x = x
        self.y = y
        self.speed = speed
        self.color = col  # (R, G, B) tuple

    def update(self):
        """Move the balloon upward and wrap around when off-screen."""
        self.y -= self.speed
        if self.y < -100:
            self.y = height + 50

    def show(self):
        """Draw the balloon using instance attributes."""
        # Draw string
        stroke(120)
        strokeWeight(1.5)
        line(self.x, self.y + 38, self.x, self.y + 95)

        # Draw balloon body
        noStroke()
        fill(self.color[0], self.color[1], self.color[2])
        ellipse(self.x, self.y, 60, 76)

        # Draw balloon knot
        triangle(self.x - 5, self.y + 42, self.x + 5, self.y + 42, self.x, self.y + 37)

# List containing 5 Balloon objects
balloons = []

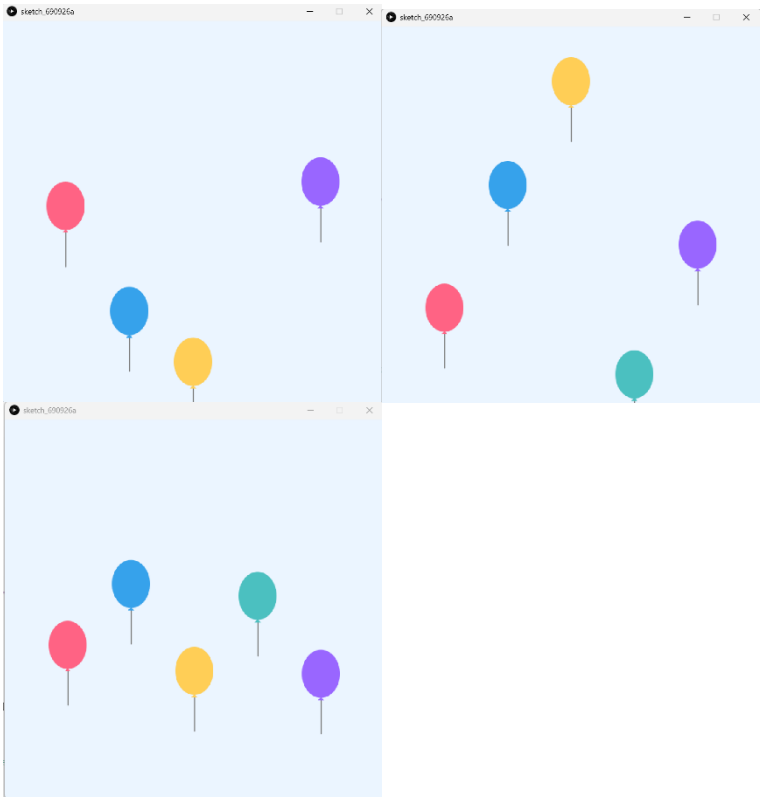
def setup():
    size(600, 600)

    # Populate the array with 5 Balloon instances
    balloons.append(Balloon(100, 600, 2.0, (255, 99, 132)))
    balloons.append(Balloon(200, 650, 3.2, (54, 162, 235)))
    balloons.append(Balloon(300, 580, 1.5, (255, 206, 86)))
    balloons.append(Balloon(400, 620, 2.8, (75, 192, 192)))
    balloons.append(Balloon(500, 670, 2.2, (153, 102, 255)))

def draw():
    background(235, 245, 255)

    # Update and display each balloon object
    for b in balloons:
        b.update()
        b.show()
```

ผลลัพธ์จากโปรแกรมทั้งสองแบบ



ข้อแตกต่าง

1. โครงสร้างและการจัดการข้อมูล (Data Structure)

- แบบไม่ใช้ Class: ใช้ List ของ Dictionary เพื่อระบุพิกัด ความเร็ว และสี ซึ่งต้องอ้างอิงผ่านคีย์ข้อความ เช่น b["y"] หรือ b["speed"] ทำให้เสี่ยงต่อการพิมพ์ชื่อคีย์ผิดพลาด (Typo) ได้ง่าย
- แบบใช้ Class: สร้างพิมพ์เขียว (Blueprint) ผ่านคลาส Balloon และสร้างอินสแตนซ์ผ่าน __init__ การเข้าถึงข้อมูลทำผ่านแอตทริบิวต์โดยตรง เช่น self.y หรือ self.speed ซึ่งมีความเป็นระเบียบและชัดเจนกว่า

2. การประมวลผลและการวาดผล

- แบบไม่ใช้ Class: ตรรกะการเคลื่อนที่ (b["y"] -= b["speed"]) และการตรวจสอบขอบจออยู่ปะปนในฟังก์ชันหลัก draw() จากนั้นจึงส่งค่าตัวแปรเข้าไปในฟังก์ชัน draw_balloon(x, y, col) อีกทอดหนึ่ง

- แบบใช้ Class: แต่ละอ็อบเจกต์จะแยกกันอย่างสมบูรณ์ เมธอด update() จัดการการเลื่อนตำแหน่งและการรีเซ็ตขอบจอ ส่วนเมธอด show() ดึงค่าสีและพิกัดของตัวเองมาร่างภาพโดยอัตโนมัติ

3. ความกระชับและความชัดเจนใน draw()

- แบบไม่ใช้ Class: ภายใน draw() จะต้องเห็นรายละเอียดขั้นตอนการคำนวณทางคณิตศาสตร์และการเรียกฟังก์ชันแสดงผลพร้อมพารามิเตอร์หลายตัว
- แบบใช้ Class: ภายใน draw() เหลือเพียงการสั่งคำสั่งตามบทบาทของวัตถุ คือ b.update() และ b.show() ทำให้ฟังก์ชันหลักสะอาดและมองเห็นภาพรวมของโปรแกรมได้ทันที

4. สามารถขยายต่อในอนาคต

- แบบไม่ใช้ Class: เหมาะสำหรับโปรเจกต์ขนาดเล็กหรือการทดลองสเก็ตช์ภาพเบื้องต้นอย่างรวดเร็ว แต่หากต้องการเพิ่มคุณสมบัติ เช่น ให้ลูกโป่งแกว่งไปมาตามแรงลม หรือแตกเมื่อคลิกเมาส์ จะต้องคอยปรับแต่ง Dictionary และฟังก์ชันแยกต่างหากหลายจุด
- แบบใช้ Class: ออกแบบมาเพื่อโปรแกรมที่มีการจำลองหลายวัตถุ (Multi-agent/Particle Simulation) หากต้องการเพิ่มพฤติกรรมใหม่ สามารถบรรจุไว้ในเมธอดของ Balloon ได้ทันที หรือนำไปสร้างลูกโป่งแบบสุ่ม นับร้อยลูกได้อย่างเป็นระบบ